

TECHNICAL DESIGN RATIONALE

Alarma

Adaptive Anti-Oversleep Destination Alarm & Emergency Safety System — the reasoning behind every technology and architecture decision.

Project: Capstone & Application Development

Program: BS Information Technology — Polytechnic University of the Philippines

Development Team: BS Information Technology Capstone Project Group

Platform: Android (.NET MAUI, .NET 9) · **Version:** 1.0.0

Document: Design & Technology Justification

1 Purpose of This Document

This companion to the README answers a single question for every major decision in Alarma: *why this, and not the alternative?* It documents the reasoning behind the platform, the architecture, the data and security choices, and the production build — so reviewers can evaluate the engineering judgement, not just the result.

Guiding principles. Three constraints shaped nearly every choice below: the app must work **offline-first** (commuters lose signal underground and on long routes), it must be **privacy-first** by law (Republic Act No. 10173, the Data Privacy Act of 2012), and it must remain **diagnosable in the field** where no debugger is attached.

2 Platform & Framework

Why .NET MAUI on .NET 9

CHOSEN: .NET MAUI

Why: A single C# codebase compiles to a native Android app with direct access to platform APIs (foreground services, `SmsManager`, Keystore) that the safety features depend on. The team's existing C#/.NET fluency meant no new language ramp-up, and .NET 9 brings mature AOT and IL-trimming used for the hardened Release build.

Over the alternatives: Flutter/React Native would have meant a new language and a plugin bridge for every native security call; native Kotlin would have delivered one platform at a higher cost with no benefit, since iOS is explicitly out of scope.

Why Android-only (API 26+)

CHOSEN: NET9.0-ANDROID

Why: The target users — Metro Manila jeepney, UV Express, and city-bus commuters — are overwhelmingly on budget and mid-range Android devices. API 26 (Android 8.0) as the floor covers the vast majority of in-use handsets while still giving access to modern foreground-service and notification APIs. iOS, web, and desktop are deliberately out of scope to keep the safety surface small and well-tested.

Why the MVC architecture

CHOSEN: MODEL-VIEW-CONTROLLER

Why: A clear separation between data models, XAML views, and controllers keeps the security-critical logic (validation, crypto, location handling) in testable, UI-independent classes. That separation is what makes the standalone xUnit test suite possible without any Android/MAUI dependency.

3 Data Storage & Persistence

Why SQLCipher over plain SQLite

CHOSEN: SQLITE-NET-PCL + SQLCIPHER (AES-256)

Why: All structured data — trip history, saved routes, emergency contacts, geocode cache, behavioral profiles — is sensitive. SQLCipher encrypts the entire database file at rest with AES-256, so a lost, stolen, or rooted device yields nothing readable. `sqlite-net-pcl` gives a clean ORM with **parameterized queries only**, eliminating SQL-injection surface.

Over the alternatives: Plain SQLite stores data in cleartext on disk; a flat JSON/Preferences store offers neither encryption nor query integrity.

Why split Preferences vs. SecureStorage

CHOSEN: PREFERENCES FOR SETTINGS, KEYSTORE FOR SECRETS

Why: Non-sensitive UI settings live in fast MAUI `Preferences` ; every cryptographic key lives in the **Android Keystore** via `SecureStorage` . No key is ever written to Preferences or hardcoded in the binary, so decompiling the APK reveals no secrets.

One key, one home. The database, backup, and black-box keys are each 256-bit, generated by `RandomNumberGenerator` , and stored only in the Keystore (slots `alarma_db_key_v1` , `alarma_backup_key_v2` , `alarma_blackbox_key_v2`). This single rule — random keys, Keystore-resident, never hardcoded — underpins the entire cryptographic story.

4 Security & Cryptography

Security was a primary design constraint, not an afterthought. The threat model assumes an attacker with physical or filesystem access to the device.

Why AES-256-GCM everywhere encryption is local

CHOSEN: AES-256-GCM (AESGCM)

Why: GCM is **authenticated** encryption — it provides confidentiality *and* integrity in one pass. Any tampering with the ciphertext, nonce, or tag makes decryption fail before a single byte is returned. Both the encrypted backup and the forensic black-box log verify the GCM tag **before** any JSON is deserialized, closing the door on data-integrity attacks (OWASP A08).

Over the alternatives: AES-CBC would require a separate HMAC step and is easy to get wrong; using the platform's `System.Security.Cryptography` avoids hand-rolled crypto entirely.

Why a Forensic Black Box Logger

CHOSEN: ENCRYPTED, KEYSTORE-BACKED CRASH LOG

Why: Faults happen in the field — on a moving jeepney at night — where no debugger or ADB exists, and a Release build emits no `Debug.WriteLine` output. The logger seals crashes with AES-256-GCM, then on next launch decrypts the prior fault to a readable report and deletes the source (consumed once). The key is preloaded at startup so the crash handler can encrypt synchronously without awaiting, and the writer never throws — there is no crash-on-crash loop.

Why validate-before-clear on restore

CHOSEN: DEFENSIVE RESTORE DESIGN

Why: A naïve restore that wipes the database first could destroy live user data if the backup is corrupt. Alarma validates **every** record — length caps, Philippine phone-number format, coordinate bounds, quantity caps — before touching the existing database. A tampered or empty backup therefore cannot silently erase a user's contacts or routes (OWASP A04).

How the controls map to OWASP

OWASP category	Why our control addresses it
A02 Cryptographic Failures	AES-256-GCM + SQLCipher AES-256; all keys random and Keystore-resident; no hardcoded keys.
A03 Injection	Parameterized SQL only; map labels via <code>JsonSerializer</code> ; search input URL-encoded; phone numbers regex-validated.
A05 Security Misconfiguration	<code>usesCleartextTraffic="false"</code> , <code>allowBackup="false"</code> , foreground service <code>Exported=false</code> .
A08 Data Integrity Failures	GCM authentication tags verified before any deserialization.
A09 Logging Failures	Encrypted forensic logging that stays readable in Release; handled exceptions routed, not swallowed.

5 Location, Mapping & Geocoding

Why an Android foreground service for tracking

CHOSEN: FOREGROUND SERVICE, EXPORTED=FALSE

Why: The destination alarm must keep computing distance even when the screen is off and the app is backgrounded — exactly what Android kills a normal background task for. A foreground `Service` with `ForegroundServiceType=TypeLocation` (required from API 34) keeps tracking alive; `Exported=false` means no other app can start or bind it. Coordinates are held in-memory as immutable values and **never logged or persisted**.

Why a PARTIAL_WAKE_LOCK

CHOSEN: SCOPED WAKE LOCK

Why: Aggressive OEM battery managers (common on the budget phones our users carry) throttle GPS delivery when the CPU sleeps, which would make the alarm miss the stop. A `PARTIAL_WAKE_LOCK` is held **only** while location providers are registered and released on stop — keeping delivery reliable without leaking the lock or draining the battery.

Why Leaflet + CartoDB in a WebView, not a native map SDK

CHOSEN: LEAFLET.JS, NO API KEY

Why: Google Maps SDK requires an API key, a billing account, and sends usage telemetry — at odds with the no-backend, privacy-first design. Leaflet with CartoDB dark tiles renders in a MAUI `WebView` under a strict Content-Security-Policy, needs **no API key and no personal identifiers**, and the dark theme suits late-night use.

Why Photon primary + Nominatim fallback + local aliases

CHOSEN: LAYERED OPEN GEOCODING

Why: Destination search is the only feature that needs the network, so it uses free, open services with no keys. Photon is fast and typo-tolerant; Nominatim is the fallback when Photon misses; and a client-side Philippine alias dictionary resolves local landmark names (e.g., colloquial stop names) that neither service indexes well. Geocode results are cached in the encrypted database to cut repeat lookups.

6 Emergency SOS

Why cellular SMS, not an internet push

CHOSEN: ANDROID SMSMANAGER

Why: An emergency is exactly when there is no Wi-Fi and no mobile data — but cellular SMS still works. Sending live coordinates directly through Android `SmsManager` means SOS functions fully offline and **never passes through any Alarma server**, so there is no backend to fail, leak, or subpoena.

Safeguards: a 2-second press-and-hold prevents accidental triggers (timer cleared on page exit), and a 30-second cooldown prevents runaway sending. The action gives a short haptic + beep confirmation but deliberately does not hijack the ringer or Do-Not-Disturb (see Section 7).

7 Capstone Audit Resolution

A focused functional and UX audit ahead of the panel defense closed twelve defects in the commute pipeline and the safety features. None of these changes alter the cryptographic story in Sections 3–4 — the SQLCipher database, the AES-256-GCM backup and black box, and the Keystore key handling are all untouched.

Infrastructure & permissions

Mandatory GPS enforcement

PRE-FLIGHT THE DEVICE LOCATION SWITCH

Why: A user can grant the app location permission yet still have the phone's master location switch off — in which case no fix ever arrives and a "trip" would track nothing. Starting a trip now checks

`ILocationService.IsLocationServiceEnabled()` first and, if location is off, blocks the start and sends the user straight to the OS location settings — the same way a navigation app refuses to guide without GPS.

Two-way permission toggles

HONOR THE OFF PATH

Why: The setup switches only ever requested permissions; flipping one off did nothing. Android offers no API to revoke a runtime permission programmatically, so OFF now tears down local state (an active tracking service and its wake lock are stopped) and routes the user to App Settings, where the OS actually allows revocation. On return, the switch re-syncs to the real status.

Alarm, tracking & state

Haversine distance with a GPS-jitter gate

NOISE-GATED PATH INTEGRAL

Why: Trip distance is summed as a true Haversine path integral between fixes. A stationary phone still emits fixes that wander a few metres, which would inflate the total, so a per-segment **jitter gate** discards any movement below the fix's own accuracy (minimum 8 m) and only advances the comparison anchor on an accepted segment — rejecting noise while still accumulating genuine slow movement.

Full state reset on trip stop

NO GHOST STATE

Why: Stopping a trip used to leave the destination set, so a stale *View Active Trip* affordance lingered on the home screen. Stop now performs a complete reset — destination and map marker cleared, search header restored, the in-memory active trip dropped, and the foreground service killed — returning the home screen to its idle state.

Monotonic, outlier-resistant escalation

PERSISTED ARRIVAL

Why: The three alarm stages are monotonic by construction (lead $\geq$ 300 m > arrival 200 m > overshoot), and arrival must now hold across consecutive fixes before it latches — so a single outlier fix can no longer fake an arrival, nor the false overshoot that would chase it. The vehicle-type selector was removed in favor of a single adaptive, speed-based lead distance.

Feedback & SOS

SMS-first SOS with a discreet cue

NO RINGER / DND HIJACK

Why: SOS previously fired the full multi-stage alarm and overrode Do-Not-Disturb — the opposite of what someone in a vulnerable situation needs. It is now strictly an SMS dispatch with a short **haptic buzz and a brief confirmation beep** so the press registers quietly. The cue respects the phone's current volume and DND, so a silenced phone stays silent.

Smaller UX fixes in the same pass. Tapping a trip notification now deep-links straight to the active-trip screen via an explicit `PendingIntent` through a `SingleTop` activity; the origin row reads "Searching for GPS..." instead of "Location unavailable" while a fix is acquired; and the emergency-contact form shows inline feedback for both validation errors and successful saves instead of failing silently.

8 Interface, Iconography & Layout Engineering

The UI is judged on a moving vehicle, one-handed, often at night. These decisions are about legibility, predictable touch targets, and assets that scale cleanly across the wide range of screen densities our commuter devices ship with.

Why Material Symbols font ligatures over raw emojis

CHOSEN: MATERIALSYMBOLSOUTLINED.TTF

Why: The interface originally leaned on raw system emojis for glyphs like search, location, and the favorite star. Those render differently on every OEM skin and Android version — wrong weight, wrong color, sometimes a low-fidelity bitmap — so the UI looked inconsistent across the exact budget handsets our users carry. We migrated every icon to native Google **Material Symbols** font ligatures, bundled as `MaterialSymbolsOutlined.ttf` and registered once in `MauiProgram.cs` under the `MaterialSymbols` font alias. A glyph is now just text (`Text="location_on"`), so it inherits font size and color, stays vector-sharp at any density, and looks identical on every device.

Over the alternatives: Per-icon PNGs would multiply asset count across density buckets and still rasterize; emojis surrender all control of weight and color to the platform.

Why the onboarding layers split background from touch targets

CHOSEN: BACK-LAYER GRID + INPUTTRANSPARENT

Why: The tutorial illustrations first shipped in an aspect-fit box, which letterboxed them with black bars and — worse — left the image's container sitting over the Skip / Next / Get Started controls, swallowing every tap. The fix isolates each illustration into the back layer of a `Grid` with `Aspect="AspectFill"` so it fills the canvas edge-to-edge with no bars, and marks it `InputTransparent="True"` so the OS ignores it entirely for hit-testing. Foreground text and the navigation controls live in later layers with an explicit high `ZIndex`, so they always render on top and keep an unobstructed touch vector.

Over the alternatives: Stacking everything in one flow layout is what caused the occlusion; absolute-positioning the buttons over an image would have re-broken on every new aspect ratio.

Why a standardized gold-yellow favorite accent

CHOSEN: MATERIAL GOLD-YELLOW #FFC107

Why: The favorite star in the saved-location list rendered in the same muted purple as the rest of the theme, so it read as decoration rather than a state the user could recognize and act on. Standardizing the star on Material Design gold-yellow (`#FFC107`), applied directly in the list item template so it colors every entry, gives a high-contrast, universally understood "favorited" signal that survives the dark theme and dim, glare-prone commute lighting.

9 Production Build Hardening

The Release configuration does double duty: it shrinks and speeds up the APK *and* it raises the cost of reverse-engineering.

Setting	Why we enabled it
<code>RunAOTCompilation</code>	Compiles managed IL to native code, so there is no IL left to decompile in the shipped APK.
<code>AndroidEnableProfiledAot</code>	Profile-guided AOT prioritizes hot paths (alarm trigger, GPS callback, geocoding) for faster cold start.
<code>PublishTrimmed / TrimMode=full</code>	Removes unreferenced code across the whole dependency closure — smaller binary, less dead-code attack surface.
<code>AndroidLinkMode=SdkOnly</code>	Strips unused SDK/AndroidX code at the DEX level.
<code>DebugSymbols=false</code>	No symbols or sequence points, so decompilers cannot map native code back to source lines.

Why `TrimmerRoots.xml` exists. The IL trimmer cannot see types created by reflection. Because the SQLite ORM instantiates model classes reflectively, the `AlarmaApp.Models` namespace and the SQLite assemblies are explicitly rooted so full trimming does not delete classes that are only ever referenced at runtime.

10 Testing Strategy

Why a separate net9.0 xUnit project

CHOSEN: `ALARMAAPP.TESTS (PLAIN NET9.0)`

Why: The security-critical logic — phone-number validation, coordinate bounding, alarm-sound whitelisting, lead-time clamping, backup restore caps, AES-GCM length checks, and the speed-based multi-stage alarm — must be verifiable without spinning up an Android emulator. Targeting plain `net9.0` with no MAUI dependency keeps the suite fast and runnable in CI, which is only possible because MVC kept that logic out of the views.

11 Summary of Decisions

Area	Choice	Core reason
Framework	.NET MAUI / .NET 9	One C# codebase, native API access, mature AOT/trimming.
Platform	Android API 26+	Matches the target commuters' devices; small safety surface.
Database	SQLCipher AES-256	All local data encrypted at rest; parameterized queries.
Secrets	Android Keystore	Random keys, never hardcoded or in Preferences.
Local crypto	AES-256-GCM	Authenticated encryption; tag verified before deserialize.
Tracking	Foreground Service	Survives backgrounding; private; coordinates never persisted.
Maps	Leaflet + CartoDB	No API key, no telemetry, privacy-first.
Geocoding	Photon + Nominatim + aliases	Free, keyless, locally aware, cached.
SOS	Cellular SMS + discreet cue	Works offline; no backend; haptic/beep only, no ringer or DND hijack.
Trip reliability	GPS enforcement + jitter-gated distance	Blocks dead trips when location is off; rejects GPS noise; clean state resets on stop.
Iconography	Material Symbols font	Vector-sharp at any density; identical across OEMs.
Onboarding layout	AspectFill back-layer + InputTransparent	Full-bleed art with unobstructed touch targets.
Favorite accent	Gold-yellow #FFC107	High-contrast, recognizable favorited state.
Release	AOT + full trim	Smaller, faster, harder to reverse-engineer.